

Agent 개발 workflow 데모

[A](#) [A](#)

1. 개발 전에 무엇을 해야할까?

AI와 함께 개발을 시작하기 전에 사람이 먼저 결정해야 할 것들이 있음

아래 결정들을 시에게 맡기면 AI가 선택할텐데 그 선택이 내 의도와 같다는 보장은 없다.

- **무엇을 개발해야할지 결정** — 기능 범위, 우선순위, 제외할 것
- **어떤 순서로 개발해야할지 결정** — 의존성 파악, 독립적으로 개발 가능한 단위 식별
- **AI에게 어떻게 시킬지 설계**
 - 어떻게 동작해야 하는가?
 - UI는 어떻게 구성되어야 하는가?
 - 완성됐다는 것을 어떻게 확인할 수 있는가?
 - 코드의 품질이 의도한 형태인가?

2. 장기 기억 — AI는 기억을 못한다

AI는 대화 세션이 바뀌면 이전 맥락을 기억하지 못한다. 그래서 **맥락을 유지하는 전략**이 필요

- **memory 기능** — Claude의 자동 메모리 요약 기능. 제한적이고 통제가 어려움
- **CLAUDE.md** — 가장 중요한 수단. 매 세션마다 AI가 읽는 '공통 지식'
- **별도 문서** — 스펙 문서, 디자인 시스템, 체크리스트 등. 필요한 시점에 참조시킴

CLAUDE.md가 없으면?

AI는 매번 다른 사람이 된다. 어제 합의한 컨벤션, 구조, 원칙이 사라짐

3. CLAUDE.md의 역할

CLAUDE.md는 **헌법**.

개발 기능 구현 방법이 아니라, 이 프로젝트에서 절대 변하지 않는 원칙들을 담는다.

- **디자인 톨거리** — 색상, 타이포, 간격 등 디자인 시스템의 핵심 변수. 상세한 스타일은 별도 파일로 분리
- **아키텍처와 디렉토리 구조** — 컴포넌트 분리 기준, 파일 위치 규칙
- **핵심 1원칙들** — "상태는 App.tsx에서 관리한다", "하드코딩 금지" 같은 프로젝트 단위 원칙 (개개인, 팀의 규칙)
- **매 세션에서 지켜야 할 공통 사항** — 커밋 메시지 형식, 코드 컨벤션, 금지 패턴

****무엇을 넣고 무엇을 빼야 할까? ****

넣어야 할 것	빼야 할 것
프로젝트 전반에 적용되는 원칙	특정 기능의 구현 방법
AI가 실수하기 쉬운 패턴의 금지 규칙	한 번만 쓰는 일회성 지시
기술 스택과 환경 정보	코드 자체

CLAUDE.md가 너무 길어지면 AI가 중요한 부분을 놓침.

가볍지만 담백하게.

규모에 따라 다르지만 50 - 200줄 미만권장.

4디자인 시스템정보

디자인 토론회 과정이 필요할 수도 있음.

- 현재 가이드 : 각 화면이 어떻게 보여야 하는가"의 결과 가이드
- 시각적 결정(이 색상, 이 간격, 이 radius)을 이름으로 굳혀서 코드 어디서나 같은 참조를 쓰게 만들기
- 일관성·변경 비용-AI/사람 협업 안정성을 동시에 잡는 것
- 계층을 나눠야 함.
 - 카테고리별
 - color, spacing, radius, typography, motion...
 - 계층별로
 - primitive, semantic, Component
 - 의도 단위로 변경 할 수 있게 분기점 만들기(semantic 계층)
 - "같은 값, 여러 의도" 상황.
 - 특정 영역 변경이 필요하면 semantic 에서 var()만 변경

```
/* 1단계: primitive.css - 값 정의 */
:root {
  --gray-100: #EEF1F5;
}

/* 2단계: semantic.css - 5개 의도가 모두 같은 gray-100을 가리킴 */
```

[<> 넓게 보기](#)[🔗 코드 복사](#)

```
:root {
  --color-surface-hover:      var(--gray-100); /*변경시 여기만 gray-200으로 변경*/
  --color-action-disabled-bg: var(--gray-100);
  --color-skeleton:          var(--gray-100);
  --color-divider-subtle:    var(--gray-100);
  --color-empty-state-bg:    var(--gray-100);
}
```

- 최종결과 예시 (`primitive.css` , `semantic.css`)
- 현재는 Primitive + Semantic 2종이면 충분

```
/* tokens/primitive.css ————— 원시 팔레트 (의미 없음, 값만) */
:root {
  /* Color - Neutral */
  --gray-0: #FFFFFF;
  --gray-50: #F5F7FA;
  --gray-100: #EEF1F5;
  --gray-200: #E1E5EB;
  --gray-400: #9AA3AE;
  --gray-700: #4A5268;
  --gray-900: #121417;

  /* Color - Brand */
  --violet-500: #7C7AED;
  --blue-700: #385FE0;

  /* Spacing scale (4의 배수) */
  --space-1: 4px; --space-2: 8px; --space-3: 12px;
  --space-4: 16px; --space-6: 24px; --space-8: 32px;
  .....
```

```
/* tokens/semantic.css ————— 역할 (어디서 어떤 의도로 쓰이나) */
:root {
  /* Surface */
  --color-surface-canvas: var(--gray-50); /* 페이지 배경 */
  --color-surface-card: var(--gray-0); /* 카드 배경 */
  --color-surface-hover: var(--gray-100); /* 셀 hover */

  /* Text */
  --color-text-primary: var(--gray-900);
  --color-text-muted: var(--gray-700);
  --color-text-on-accent: var(--gray-0); /* 진한 배경 위 흰 글씨 */

  /* Border */
  --color-border-default: var(--gray-200);

  /* Action / Selection */
  --color-action-selected-bg: var(--blue-700); /* 활성 카테고리 탭 */
  --color-action-accent-bg: var(--violet-500); /* 카운트 배지 */
  --color-focus-ring: var(--violet-500);
  .....
```

5. 워크플로우

개발의 흐름은 규모와 제약조건에 따라 다름.

AI쓰기 전에도 이 흐름은 상황마다 팀마다 다 달랐음.

AI를 사용하면서 기존 워크플로우에서 어떻게 설계하고 통제하며 검증할지가 추가되었다고 볼 수 있음.

5-1. 요구사항 수집

- 구현해야 할 것과 하지 않을 것을 명확히 구분
- 기획 문서(PDF, 피그마 등)가 있다면 그대로 AI에게 넘기지 말고 **직접 읽고 해석할 것**

5-2. 스펙 문서화

- 요구사항을 마크다운 문서로 정리
- **빠진 스펙을 찾아내는 것이 핵심** — AI에게 "이 스펙에서 구현하려면 더 필요한 정보가 무엇인가?"라고 역으로 질문시킬 것
- 누락된 스펙은 AI가 임의로 채운다. 그 결과가 의도와 다를 가능성이 높음

5-3. 스펙 나누기

- **적당한 크기**란? — 하나의 기능이 독립적으로 동작 가능한 단위
- 기능별로 의미있게 나누기. 나눠진 기능이 서비스로서 '의미'를 가지면 좋음
- 동작 가능한 상태로 나눠지면 더 좋음 — 중간중간 확인이 가능해짐
- 너무 크면: AI가 한 번에 너무 많은 코드를 생성 → 검증 불가, 커밋 단위 붕괴
- 너무 작으면: 컨텍스트 단절, 파편화된 구현

5-4. 하나씩 개발하기

🕒 feature branch로 격리

- 기능 단위로 브랜치를 만들어 개발. main이 항상 동작 가능한 상태를 유지

🕒 개발 전 구조 설계

- 이 기능에 필요한 컴포넌트, 상태, 데이터 흐름을 먼저 그리

- 시에게 코드부터 짜달라고 하지 말고 "이 기능의 구조를 먼저 제안해줘"로 시작

⑩ **체크리스트 작성 (완료 조건)**

- 엣지케이스 포함한 완료 조건을 먼저 정의
- "이 기능이 완성됐다"는 기준이 없으면 AI도 사람도 언제 멈춰야 할지 모름

⑪ **개발 진행**

- 하위 태스크(있다면) 순서대로 구현
- 한 번에 너무 많이 요청하지 않기 — incremental하게

⑫ **리팩토링 — 왜 지금 해야 하는가?**

- AI는 동작하는 코드를 빠르게 만들지만, 구조는 누락될수록 나빠짐
- 다음 기능을 엮기 전에 정리하지 않으면 기술 부채가 복리로 쌓임
- **리팩토링 기준**: 중복 제거 / 하드코딩 상수화 / 컴포넌트 단일 책임 / 네이밍 명확화

⑬ **커밋**

- 규칙 기반 커밋 — 커밋 메시지 형식은 CLAUDE.md에 정의해두고 AI가 자동 작성하게 구성 가능
- 커밋 단위: 기능 하나 + 그에 딸린 리팩토링까지

5-5. 검수

AI 없이 개발했다면 어떤 것을 확인했을까? 그 기준을 그대로 적용한다.

- **디자인 / UX** — 스펙과 픽셀 단위 비교, 인터랙션 동작 확인
- **컨벤션 준수** — CLAUDE.md 규칙 위반 여부
- **보안** — 환경변수 노출, fetch URL 하드코딩, 인증 누락
- **재사용성** — 하드코딩된 매직 넘버, 중복 함수, 분리되지 않은 스타일 로직
- **성능** — 보이지 않는 요소까지 DOM에 렌더링하고 있지는 않은가?
- **배포 환경** — 개발 환경에서만 동작하는 코드가 없는가? (`fetch('/src/data/...')` 같은 패턴)

6. 내가 쓰는 도구의 Best Practices

AI Agent 도구는 다양하다

Claude Code, Gemini CLI, Codex CLI, Cursor 등. 도구마다 특성이 다르고, 잘 쓰는 방법도 다르다.

- **도구별 특성 파악** — 토큰 제한, 파일 읽기 방식, 컨텍스트 유지 방식의 차이
- **내가 잘 활용하고 있는지 점검** — 내 워크플로우를 AI에게 설명하고 "더 효율적인 방법이 있는가?" 피드백 받기
- **도구를 바꿔야 할 시점** — 토큰 소진, 기능 한계, 반복 실수 패턴이 나타날 때

7. 활용하며 배우기

AI를 쓴다고 실력이 늘지는 않는다. **AI가 선택한 방식을 이해하고 검증하는 과정**에서 실력이 는다.

- **AI가 선택한 방식에 질문하기** — "왜 이 방식을 선택했나? 다른 방법은 없나?"
- **이해될 때까지 토론하기** — 동의하지 않으면 동의할 때까지 물어볼일 것. AI는 설득당하지 않으면 계속 설명한다
- **다른 관점으로 비판하기** — "이 코드의 단점을 말해줘", "시니어 개발자라면 이 코드를 어떻게 볼까?"
- **같은 주제로 동료와 토론** — AI와의 대화는 혼자다. 사람과의 토론에서 나오는 것이 다름

8. 생각해볼 것들

Agent를 쓰면 개발이 더 빨라지는가?

빨라지는 것과 빨라지지 않는 것이 있다. 코드 생성은 빠르다. 하지만 검증, 리팩토링, 디버깅은 오히려 시간 이 더 걸릴 수 있다. "빠르다"는 느낌이 착각일 수 있음.

더 품질이 좋아지는가?

AI는 동작하는 코드를 만들지만, 좋은 구조를 만들지는 않는다. 품질은 사람이 개입한 만큼 올라간다.

초기 비용 vs 워크플로우 완성 후 비용

- 처음 쓸 때는 설정, CLAUDE.md 작성, 워크플로우 설계 비용이 크다
- 워크플로우가 완성되면 반복 비용이 급격히 줄어든다
- 초기 비용을 아끼려고 워크플로우 없이 쓰면, 매번 초기 비용을 낸다

학습 완료

< 이전글 Agent 기반 개발 방법 시작하기

목록으로



v130.32

코드스쿼드(대표자: 김정) 주소: 강남구 역삼동 836-24 역삼빌딩 4층 전화: 070-4117-1005
사업자등록번호: 676-88-00504 통신판매신고: 제 2017-서울강남-04281호 대표 이메일: yoda@codesquad.kr
Copyright © 2019-2026 코드스쿼드. All rights reserved

자주 묻는 질문 의견 보내기 환불 규정